Efficiency and Caching Solutions

Computer Science Mentors 61A

October 26, 2026 – October 30, 2026

1 Efficiency

Efficiency: avoiding repeated work

A cache stores results so a later request can reuse them. Memoization is caching a function's return value for each input. On a *cache miss*, compute and store the result; on a *cache hit*, return the stored result.

```
def memo(f):
    cache = {}
    def remembered(x):
        if x not in cache:
            cache[x] = f(x)
        return cache[x]
    return remembered
```

Each call to `memo` creates a new dictionary. The returned function keeps that dictionary across calls. Putting `cache = {}` inside `remembered` would discard earlier results on every call. Use membership testing: a stored result of 0 or `False` is still a cached result. This version requires hashable inputs, such as integers.

Caching works when the same input should produce the same result. A cache hit skips the original function body, including its printing or other side effects. If a result depends on changing external state, reusing it can be incorrect. Caching also uses memory to store results.

Fibonacci and repeated calls

Tree-recursive Fibonacci repeats calls on smaller inputs. For example, computing `fib(5)` needs the results for inputs 4 and 3. Computing the result for input 4 needs input 3 again. With memoization, every recursive call checks a shared dictionary keyed by the argument. Each `fib(k)` is computed only once; later requests are just lookups. Saving only the final answer to the outermost call would not prevent repeated work within that computation.

List operations differ in cost

- **Fast list-building choices:** `s.append(x)` adds to the end without shifting existing elements. List comprehensions also build results efficiently, evaluating their expression once per element. Appending occasionally needs more storage, but does not copy the whole list every time.
- **Fast replacement:** `s[k] = k * k` replaces the element at an existing index without shifting other elements or copying the list. It does not grow the list: index `k` must already be valid.
- **Slow when repeated:** `s.insert(0, x)` shifts existing elements to make room at the front. `s = s + [x]` creates a new list, copying the growing old list every time.

Created by Ana Abell, Tien-Yu Hsu, Cadence Loh, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

Explain what work is repeated or avoided, rather than giving a runtime formula.

1. Consider tree-recursive Fibonacci for non-negative integers.

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

(a) Draw the calls made by `fib(4)`. Which inputs occur more than once, and what does the original call return?

```
fib(4)
  fib(3)
    fib(2)
      fib(1)
      fib(0)
    fib(1)
  fib(2)
    fib(1)
    fib(0)
```

Inputs 2, 1, and 0 repeat. The result is 3.

(b) Suppose every call checks the same cache and saves its result. What should the keys and values be? When the second request for `fib(2)` occurs, what work is avoided?

Keys are input integers; values are their Fibonacci numbers. The first request stores 1 under key 2. The second returns that result without calling `fib(1)` or `fib(0)` again. Each distinct Fibonacci result is computed once, even if requested repeatedly.

2. Complete `make_fib` so the returned function remembers Fibonacci results across calls. Assume non-negative integer inputs.

```
def make_fib():
    cache = {0: 0, 1: 1}
    def fib(n):
        if _____________________________:
            cache[n] = _____________________________
        return _____________________________
    return fib
```

```
def make_fib():
    cache = {0: 0, 1: 1}
    def fib(n):
        if n not in cache:
            cache[n] = fib(n - 1) + fib(n - 2)
        return cache[n]
    return fib
```

(a) Execute these calls in order. Give each return value and the new keys added to the cache during each call.

```
f = make_fib()
f(5)
f(5)
f(6)
```

The first call returns 5 and adds keys 2, 3, 4, 5 in that order. The second returns 5 and adds no keys: it is a cache hit. The third returns 8 and adds only key 6; values for 5 and 4 are already stored.

(b) Why is the dictionary created outside `fib`? Would `g = make_fib()` share `f`'s saved results?

The returned function retains the same dictionary across recursive and later calls. Creating it inside `fib` would reset it on each call. Calling `make_fib` again creates a separate dictionary, so `g` does not share `f`'s results.

3. Compare these ways of building a list of greetings from a list of names.

```
def greetings_a(names):
    return ['Hi, ' + name for name in names]

def greetings_b(names):
    s = []
    for name in names:
        s.append('Hi, ' + name)
    return s

def greetings_c(names):
    s = []
    for name in names:
        s = s + ['Hi, ' + name]
    return s

def greetings_d(names):
    s = []
    for name in names:
        s.insert(0, 'Hi, ' + name)
    return s
```

(a) What does each function return for `names = ['Ada', 'Bo', 'Cam']`?

`greetings_a` (comprehension), `greetings_b` (append), and `greetings_c` (concatenation) all keep the input order and return:

`['Hi, Ada', 'Hi, Bo', 'Hi, Cam']`

`greetings_d` (front insertion) reverses the order because each new greeting is inserted before the previous greetings. It returns:

`['Hi, Cam', 'Hi, Bo', 'Hi, Ada']`

(b) Which two avoid repeatedly copying or shifting the growing list? Explain the extra work in the other two, without runtime notation. Focus on building the list; all four construct the same greeting strings.

- `greetings_a` uses a list comprehension to build the result without repeatedly copying the growing list.
- `greetings_b` uses `append` to add each greeting at the end without shifting existing elements. It may occasionally resize its storage, but does not copy the whole list on every append.
- `greetings_c` uses `s = s + [...]` to construct a new list each iteration, copying all the previous greetings into it before adding the new greeting.
- `greetings_d` uses `s.insert(0, ...)` to insert at the front, shifting all existing greetings to make room.

(c) Rewrite `greetings_d` with a list comprehension that returns the same elements in the same order, without inserting at the front.

Visit the names from the last index to the first, so the comprehension produces the same reversed order without inserting at the front.

```
def greetings_d(names):
    return ['Hi, ' + names[i]
            for i in range(len(names) - 1, -1, -1)]
```

4. [Exam Level] Implement the `Poll` class and the `tally` function, which takes a choice `c` and returns a list describing the number of votes for `c`. This list contains pairs, each with a name and the number of times `vote` was called on that `choice` at the `Poll` with that name. Pairs can be in any order. Assume all `Poll` instances have distinct names. Hint: the dictionary `get(key, default)` method returns the value for a `key` if it appears in the dictionary and `default` otherwise. [Adapted from CS61A Fa18 Midterm 2 Q5(a)]

```
class Poll:
    s = []

    def __init__(self, n):

        self.name = _____________________________________________

        self.votes = {}

        ________________________________________________________________

    def vote(self, choice):

        self._____________________________ = _____________________________________


def tally(c):
    """Tally all votes for a choice c as a list of (poll name, vote count)
       pairs.
    >>> a, b, c = Poll('A'), Poll('B'), Poll('C')
    >>> c.vote('dog')
    >>> a.vote('dog')
    >>> a.vote('cat')
    >>> b.vote('cat')
    >>> a.vote('dog')
    >>> tally('dog')
    [('A', 2), ('C', 1)]
    >>> tally('cat')
    [('A', 1), ('B', 1)]
    """

    return ____________________________________________________________
```

```
class Poll:
    s = []
    def __init__(self, n):
        self.name = n
        self.votes = {}
        Poll.s.append(self)
    def vote(self, choice):
        self.votes[choice] = self.votes.get(choice, 0) + 1

def tally(c):
    return [(p.name, p.votes[c]) for p in Poll.s if c in p.votes]
```

5. Given a list of integers `lst`, return the maximum sum of a subset of size `n`. If `n` is greater than or equal to the length of `lst`, just return the sum of the elements `lst`.

```
def max_subset_sum(lst, n):
    """
    >>> max_subset_sum([1, 2, 3, 4], 2)
    7
    >>> max_subset_sum([1, 4, 2, 0, 6], 3)
    12
    """

    if _____________________________:

        _____________________________

    elif _____________________________:

        _____________________________

    with_elem = _____________________________ + __________

    without_elem = _____________________________________

    return _____________________________________________
```

```
def max_subset_sum(lst, n):
    if n == 0:
        return 0
    elif len(lst) <= n:
        return sum(lst)
    with_elem = max_subset_sum(lst[1:], n - 1) + lst[0]
    without_elem = max_subset_sum(lst[1:], n)
    return max(with_elem, without_elem)
```

6. What Would Python Display? Expressions are run in the same environment.

```
class SkipMachine:
    skip = 1
    def __init__(self, n=2):
        self.skip = n + SkipMachine.skip

    def generate(self):
        current = SkipMachine.skip
        while True:
            yield current
            current += self.skip
            SkipMachine.skip += 1

p = SkipMachine()
twos = p.generate()
SkipMachine.skip += 1
twos2 = p.generate()
threes = SkipMachine(3).generate()
```

(a) `next(twos)`

2

(b) `next(threes)`

2

(c) `next(twos)`

5

(d) `next(twos)`

8

(e) `next(threes)`

7

(f) `next(twos2)`

5